

Computação Digital

Laboratório 1

Felipe Calliari
<calliari@puc-rio.br>

1 Introdução

Este é o nosso primeiro projeto em VHDL e o nosso objetivo é: ter o primeiro contato com o Xilinx ISE Project Navigator, escrever um código concorrente – circuito combinacional, verificar a concorrência em hardware, instanciar outras entidades como componentes de um projeto maior e a configuração da FPGA através do iMPACT.

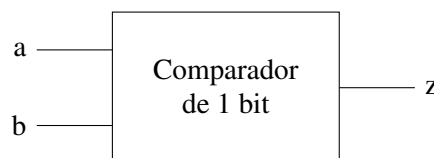
2 Comparador Binário

A tarefa é criar um comparador binário de 2 bits – isto é, duas entradas de 2 bits, mas iremos fazer isso por partes.

2.1 Comparador de 1 bit

2.1.1 Exercício 1

Crie um novo projeto no ISE. Crie a entidade de um comparador binário de 1 bit, considere duas entradas e uma saída do tipo STD_LOGIC. A saída será ‘1’ sempre que as entradas forem iguais.



Após escrever o código, verifique que este não apresente erros ao rodar “Check Syntax”, e sintetize (*synthesize*) seu código. Verifique:

- No *Summary*, verifique número de LUTs utilizadas e disponíveis e o número de IOBs utilizadas e disponíveis.
- Navegue em *Detailed Report*, veja o *delay* do circuito.
- Em *Synthesize*, analise tanto o *RTL Schematic* quanto o *Technology Schematic*. Investigue as diferenças e relate aqui. Pesquise.

2.1.2 Exercício 2

Crie um *testbench* para testar o código. Remova o código relacionado à *clock* do *testbench* gerado pelo ISE. O teste deve testar todas as possíveis combinações de entradas (00, 01, 10 e 11). Apresente os resultados (captura de tela do *waveform*) e explique o seu funcionamento.

2.1.3 Exercício 3

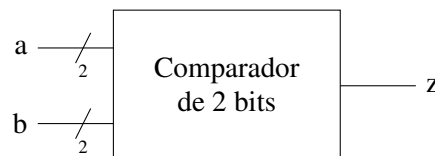
É hora de implementar! Acesse o [Spartan-3E FPGA Starter Kit Board User Guide](#), leia sobre “*slide switches*” e “*discrete leds*” no capítulo 2 do manual. Depois:

- Crie um *User Constraint File* no projeto.
- Utilize os *slide switches* SW<0> e SW<1> como entradas ‘a’ e ‘b’, respectivamente.
- Utilize o LED<0> como saída ‘z’.

2.2 Comparador de 2 bits

2.2.1 Exercício 1

Crie um novo projeto no ISE. Crie a entidade de um comparador binário de 2 bits, utilize STD_LOGIC_VECTOR para as entradas e STD_LOGIC para a saída. A saída será ‘1’ sempre que as entradas forem iguais, por exemplo: “10” = “10”.



Você não irá escrever o código para um comparador de 2 bits! Você instanciará o “comparador de 1 bit” criado na primeira parte do projeto usando a estrutura abaixo, este código será inserido entre ***architecture begin e end rtl;***

```

1  unit_label : entity lib_name.entity_name (arch_name)
2  port map(
3      port_signal => actual_signal,
4      port_signal => actual_signal,
5      -- ...
6      port_signal => actual_signal
7  );
  
```

A primeira parte (*unit_label*) é um “apelido” para o uso desta entidade que está sendo evocada. A biblioteca padrão (*lib_name*) é *work*, a entidade (*entity_name*) e a arquitetura (*arch_name*) são as que vocês criaram na primeira parte do projeto. Dentro de ***port map*** temos, no lado esquerdo, o nome das portas do componente e, no lado direito, o nome dos sinais para os quais as saídas serão atribuídas.

Exemplo:

```

1  comparador_do_primeiro_bit : entity work.comp_bit (Behavioral)
2  port map(
3      a => a(0),
4      b => b(0),
5      z => compl
6  );
  
```

Após realizar esta etapa, rode “Check Syntax”, e sintetize (*synthesize*) seu código. Verifique:

- No *Summary*, verifique número de LUTs utilizadas e disponíveis e o número de IOBs utilizadas e disponíveis.
- Navegue em *Detailed Report*, veja o *delay* do circuito.
- Em *Synthesize*, analise tanto o *RTL Schematic* quanto o *Technology Schematic*. Investigue as diferenças e relate aqui. Pesquise.

2.2.2 Exercício 2

Crie um novo *testbench* para testar o código. Remova o código relacionado à *clock* do *testbench* gerado pelo ISE. O teste deve testar todas as possíveis combinações de entradas – são 16 combinações. Apresente os resultados (captura de tela do *waveform*) e explique o seu funcionamento.

Dica: Utilize a estrutura **for loop** no *testbench*:

```
1  -- Stimulus process
2  stim_proc : process
3  begin
4      for I_A in 0 to 15 loop
5          a <= std_logic_vector(to_unsigned(I_A, 4));
6          for I_B in 0 to 15 loop
7              b <= std_logic_vector(to_unsigned(I_B, 4));
8              wait for 10 ns;
9          end loop;
10     end loop;
11     wait;
12 end process;
```

2.2.3 Exercício 3

Implemente este comparador! Acesse o [Spartan-3E FPGA Starter Kit Board User Guide](#) para copiar as definições dos *constraints*. Depois:

- Crie um *User Constraint File* no projeto.
- Utilize os *slide switches* para as entradas A e B.
- Utilize o LED<0> como saída 'z'.

3 Relatório

Os projetos devem ser realizados **individualmente** ou em **duplas**.

O relatório deve ser entregue no formato PDF, junto com a pasta do projeto e as bibliotecas (se houver) em um **arquivo ZIP** até a data de entrega pelo **Moodle**.

Antes de zipar o projeto apague os arquivos de simulação, relatórios, etc. Para isto, acesse *Project* → *Cleanup Project Files...*, e depois, *Project* → *Archive...*.

